

**WYDZIAŁ TELEKOMUNIKACJI, INFORMATYKI
I ELEKTROTECHNIKI**

[Programowanie urządzeń mobilnych](#) [05-IST-PAM-SP5]-laboratorium

Lab (Nr_10) Temat: **Aplikacja Kotlin cz 2**



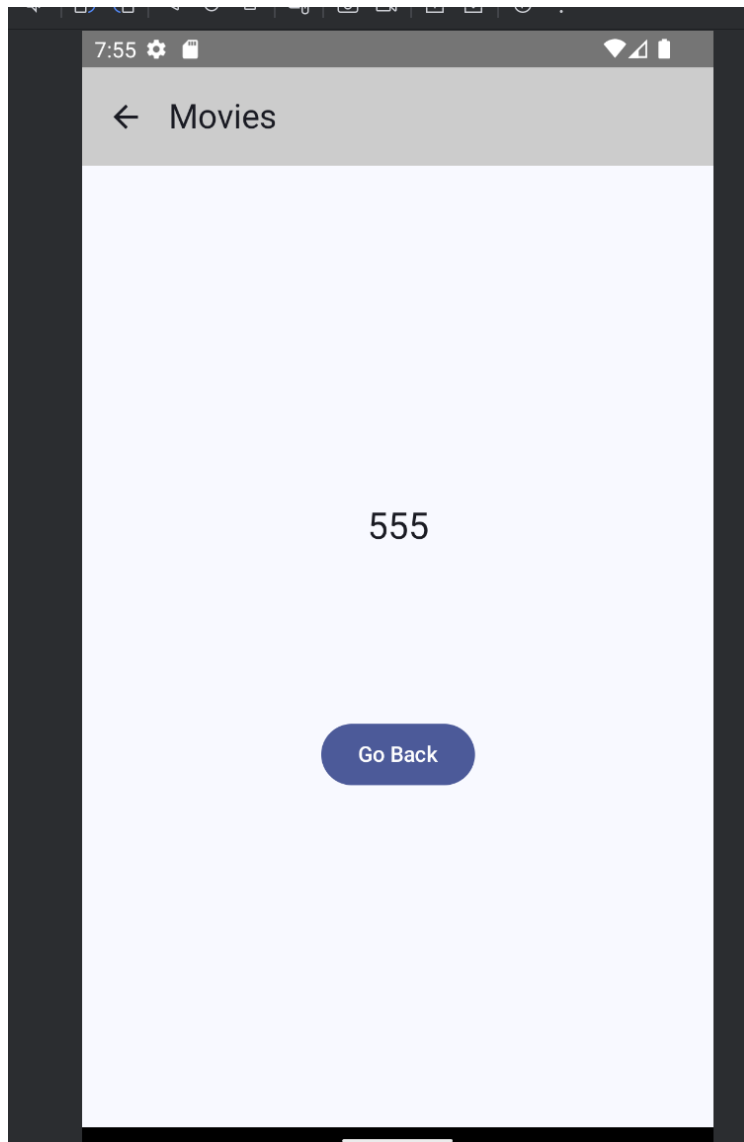
Cel:

Rozwinąć aplikację z części 1 (Aplikacja Kotlin) wyświetlającą zawartość listy z elementami String na ekranie Smartfonu z wykorzystaniem architektury Model-widok-Kontroler.

Rozwinięcie ma polegać na stworzeniu funkcji `DetailsScreen` w pakiecie `details`

`DetailsScreen` to Composable ekran w aplikacji Android, który:

- Wyświetla pasek aplikacji (`TopAppBar`) z tytułem i ikoną powrotu.
- Pokazuje przekazane dane o filmie (`movieData`) w centralnej części ekranu.
- Dodaje kilka odstępów (`Spacer`).
- Udostępnia przycisk „Go Back”, który cofa użytkownika w nawigacji (`navController.popBackStack()`).



Widok `DetailsScreen`

```

package pbs.edu.kurs3.screens.details
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import androidx.compose.material3.SmallTopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.material3.Text
@OptIn(markerClass = ExperimentalMaterial3Api::class) 2 Usages
@Composable
fun DetailsScreen(navController: NavController, movieData: String?) {
    Scaffold(
        topBar = { SmallTopAppBar(
            title = { Text(text = "Movies") },
            navigationIcon = {
                IconButton(onClick = { navController.popBackStack() }) {
                    Icon(imageVector = Icons.Default.ArrowBack, contentDescription = "Back")
                }
            },
            colors = TopAppBarDefaults.smallTopAppBarColors(containerColor = Color.LightGray)
        )
    },
        { it -> Column(modifier = Modifier.padding(paddingValues = it))
    } { this: ColumnScope
        Surface(
            modifier = Modifier
                .fillMaxHeight().fillMaxWidth()
        ) {
            Column(horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center) {
                this: ColumnScope
                Text(text = movieData.toString(), style = MaterialTheme.typography.headlineSmall)
                Spacer(modifier = Modifier.height(height = 23.dp))
                Spacer(modifier = Modifier.height(height = 23.dp))
                Spacer(modifier = Modifier.height(height = 63.dp))
                Button(onClick = { navController.popBackStack() }) { this: RowScope
                    Text(text = "Go Back")
                }
            }
        }
    }
}

```

Kod Funkcji DetailsScreen

To prosta aplikacja filmowa z:

- motywem (Kurs2Theme),
- ekranem głównym (HomeScreen) z listą filmów,
- przygotowaną nawigacją do szczegółów (DetailsScreen),
- komponentami UI (MovieRow) budowanymi w Jetpack Compose.

Struktura aplikacji

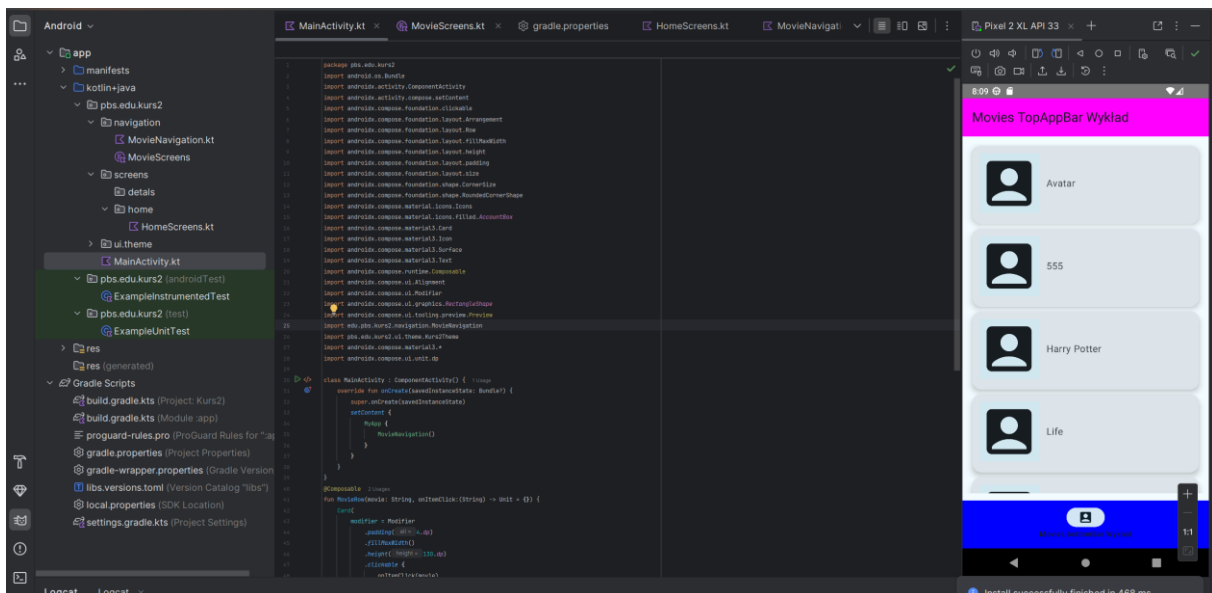
1. MainActivity

- Główna aktywność uruchamiająca aplikację.
- W metodzie onCreate wywołuje setContent, gdzie osadzony jest motyw (MyApp) i nawigacja (MovieNavigation).

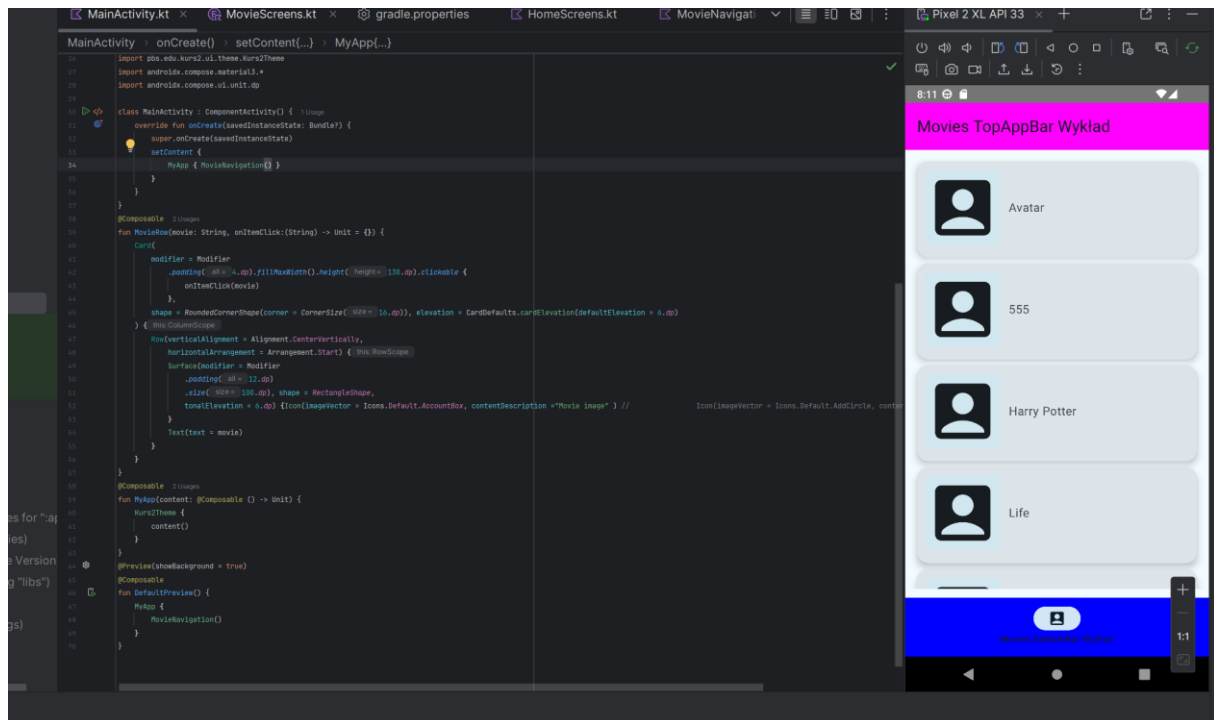
2. MyApp

- Funkcja opakowująca całą aplikację w motyw (Kurs2Theme).

- Dzięki temu wszystkie komponenty dziedziczą styl (kolory, typografia).
- ### 3. MovieRow
- Komponent UI wyświetlający pojedynczy film w postaci karty:
 - Ikona (AccountBox) jako miniatura.
 - Tekst z tytułem filmu.
 - Obsługa kliknięcia (onItemClick) - przygotowana do nawigacji do szczegółów.
- ## ◆ Ekran
- ### 1. HomeScreen
- Zbudowany w oparciu o Scaffold:
 - TopAppBar - pasek górny z tytułem.
 - BottomAppBar - pasek dolny z podpisem.
 - Body - główna treść ekranu (MainContent).
 - Tworzy spójny układ aplikacji.
- ### 2. MainContent
- Wyświetla listę filmów (moviesList) w LazyColumn.
 - Każdy element listy to MovieRow.
 - Kliknięcie w film może:
 - zapisać zdarzenie w logach (Log.d),
 - albo uruchomić nawigację do innego ekranu (DetailsScreen).
- ### 3. DetailsScreen to Composable ekran w aplikacji Android
- ### 4. ◆ Nawigacja
- Zdefiniowana w enum class MovieScreens:
 - HomeScreen
 - DetailsScreen
 - Funkcja fromRoute pozwala rozpoznać ekran na podstawie trasy (np. "DetailsScreen/Avatar" → DetailsScreen).
 - Dzięki temu aplikacja może przechodzić między ekranami i przekazywać parametry (np. tytuł filmu).



Widok drzewa projektu aplikacji. Klasa MainActivity cz1



Klasa MainActivity cz 2

MainActivity

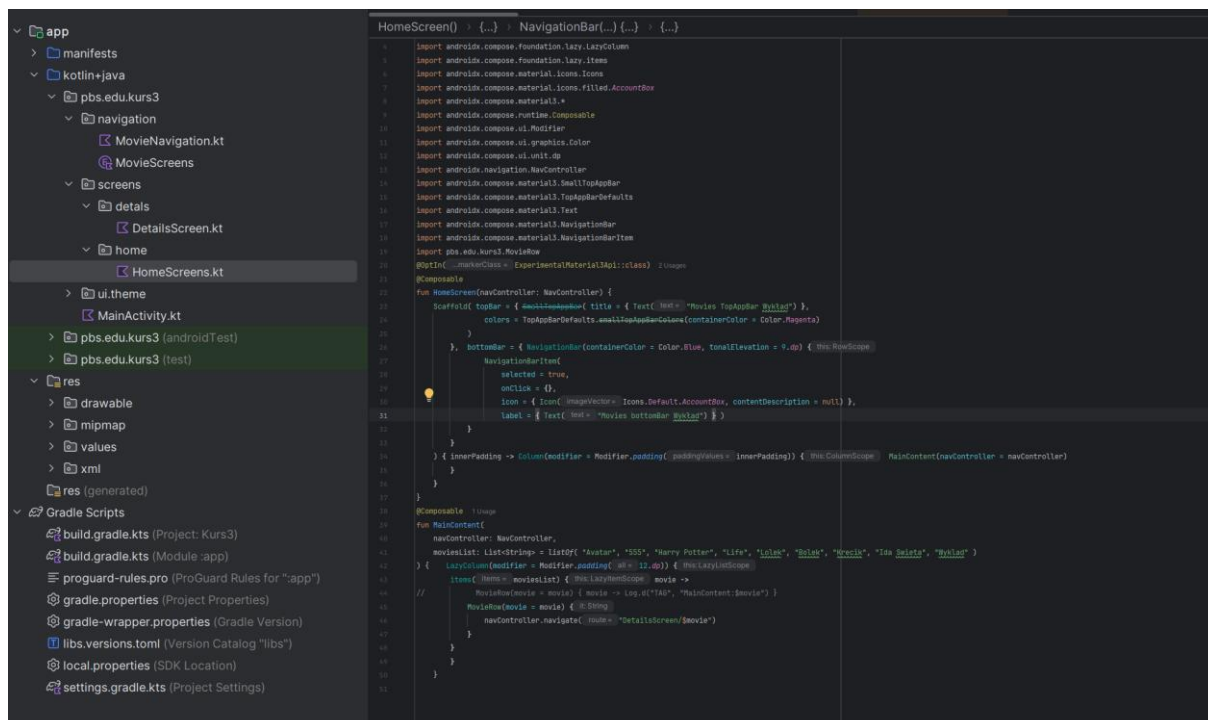
- Klasa MainActivity dziedziczy po AppCompatActivity.
- W metodzie onCreate wywoływana jest funkcja setContent, która uruchamia kompozycje (Composable functions).
- Cała aplikacja jest osadzona w funkcji MyApp, która nakłada motyw (Kurs2Theme) i uruchamia nawigację (MovieNavigation).

MovieRow

- Jest to komponent UI wyświetlający pojedynczy wiersz z filmem.
- Składa się z:
 - Card – karta z zaokrąglonymi rogami i cieniem (elevation).
 - Row – układ poziomy, w którym znajdują się:
 - Surface z ikoną (Icons.Default.AccountBox) – pełni rolę miniatury filmu.
 - Text – wyświetla tytuł filmu.
- Cała karta jest klikalna – po kliknięciu wywołuje onItemClick(movie), czyli przekazuje nazwę filmu do funkcji obsługującej zdarzenie.

MyApp

- Funkcja opakowująca całą aplikację w motyw (Kurs2Theme).
- Dzięki temu wszystkie komponenty dziedziczą styl (np. kolory, typografię).
- ◆ DefaultPreview
 - Funkcja oznaczona adnotacją @Preview. Pozwala podejrzeć wygląd aplikacji w Android Studio bez uruchamiania jej na emulatorze.
 - Wywołuje MovieNavigation, czyli pokazuje przykładowy ekran w edytorze.



Funkcija HomeScreen

HomeScreen

- Scaffold – główny kontener ekranu, który pozwala łatwo zdefiniować:
 - TopAppBar – pasek górny z tytułem "Movies TopAppBar Wykład".
 - Ma kolor magenta i cień (elevation = 7.dp),
 - BottomAppBar – pasek dolny z tekstem "Movies bottomBar Wykład".
 - Ma kolor niebieski i cień (elevation = 9.dp).
- W body ({ it -> Column(...) }) osadzony jest główny widok ekranu – MainContent.

Dzięki Scaffold aplikacja ma spójny układ: pasek górny, dolny i treść w środku.

MainContent

Przyjmuje NavController (do obsługi nawigacji między ekranami).

Definiuje listę filmów (moviesList) jako przykładowe dane:

kotlin

```
listOf("Avatarc", "555", "Harry PotterX", "Life", "Lolek", "Bolek",
"Krecik", "Ida Świeta", "Wykład")
```

Wewnątrz Column znajduje się LazyColumn - czyli przewijalna lista.

Dla każdego elementu listy wywoływana jest funkcja `MovieRow`, która:

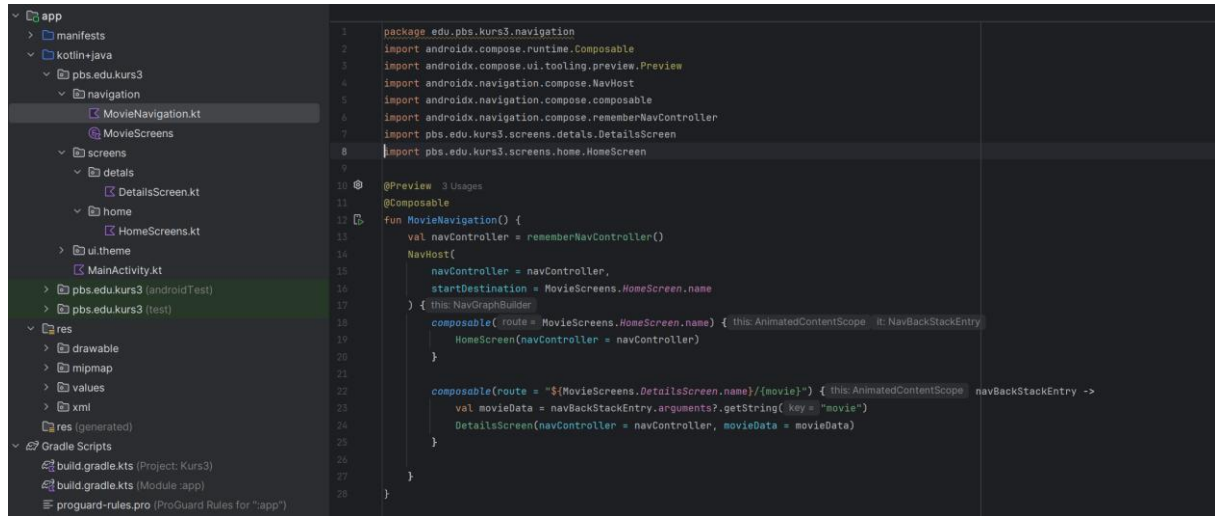
Wyświetla kartę z ikoną i tytułem filmu.

Obsługuje kliknięcia:

```
Aktualnie: MovieRow(movie = movie) {
    navController.navigate("DetailsScreen/$movie")
}
```

Do HomeScreen z parametrem filmu.

Do DetailsScreen z parametrem filmu.



Klasa `MovieNavigation`

MainContent

- Przyjmuje `NavController` (do obsługi nawigacji między ekranami).
- Definiuje listę filmów (`moviesList`) jako przykładowe dane:

kotlin

```
listOf("Avatarc", "555", "Harry PotterX", "Life", "Lolek", "Bolek", "Krecik", "Idą Święta", "Wykład")
```

- Wewnątrz `Column` znajduje się `LazyColumn` – czyli przewijalna lista.
- Dla każdego elementu listy wywoływana jest funkcja `MovieRow`, która:
 - Wyświetla kartę z ikoną i tytułem filmu.
 - Obsługuje kliknięcie:
 - Aktualnie: `Log.d("TAG", "MainContent:$movie")` – zapis do logów.
 - Zakomentowane przykłady pokazują, że można zamiast tego uruchomić **nawigację**:
 - Do `HomeScreen` z parametrem filmu,
 - Do `DetailsScreen` z parametrem filmu.

```

package edu.pbs.kurs3.navigation

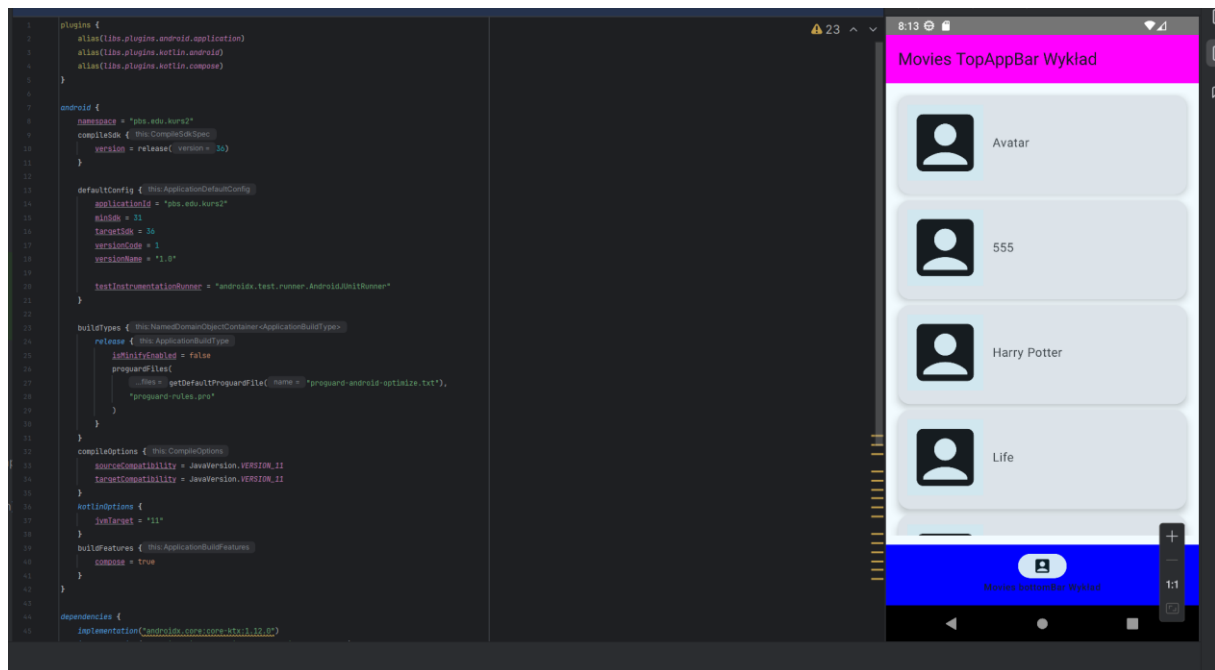
enum class MovieScreens { 4 Usages
    HomeScreen, 5 Usages
    DetailsScreen; 3 Usages
    companion object{
        fun fromRoute(route:String?):MovieScreens=
            when(route?.substringBefore(delimiter = "/"))
            {
                HomeScreen.name ->HomeScreen
                DetailsScreen.name ->DetailsScreen
                null->HomeScreen
                else -> throw IllegalArgumentException( s = "Route $route is not recognize")
            }
    }
}

```

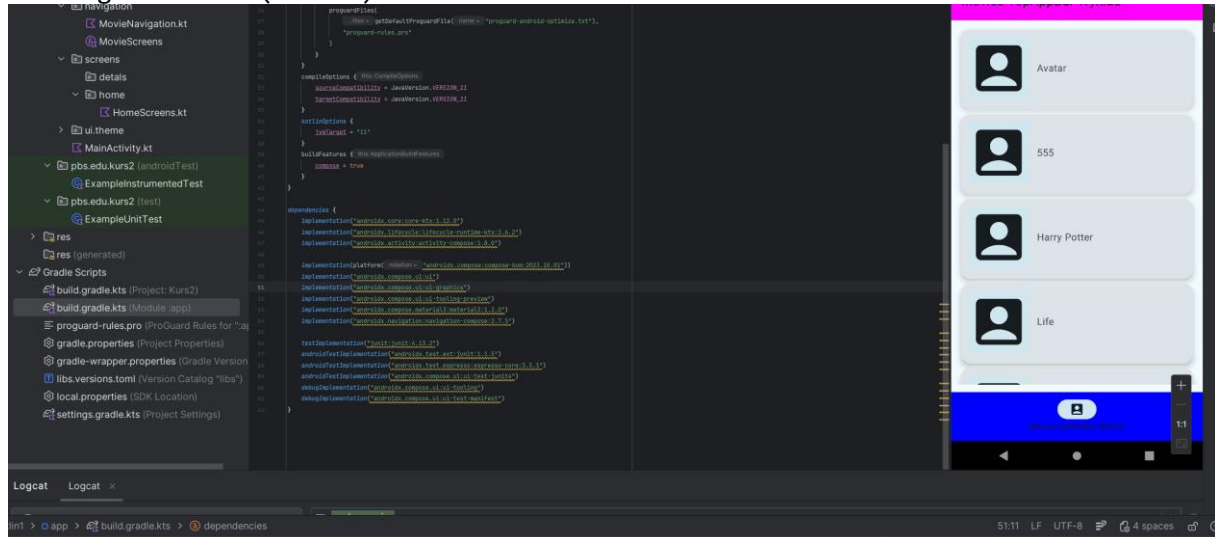
Klasa MovieScreens

enum class MovieScreens

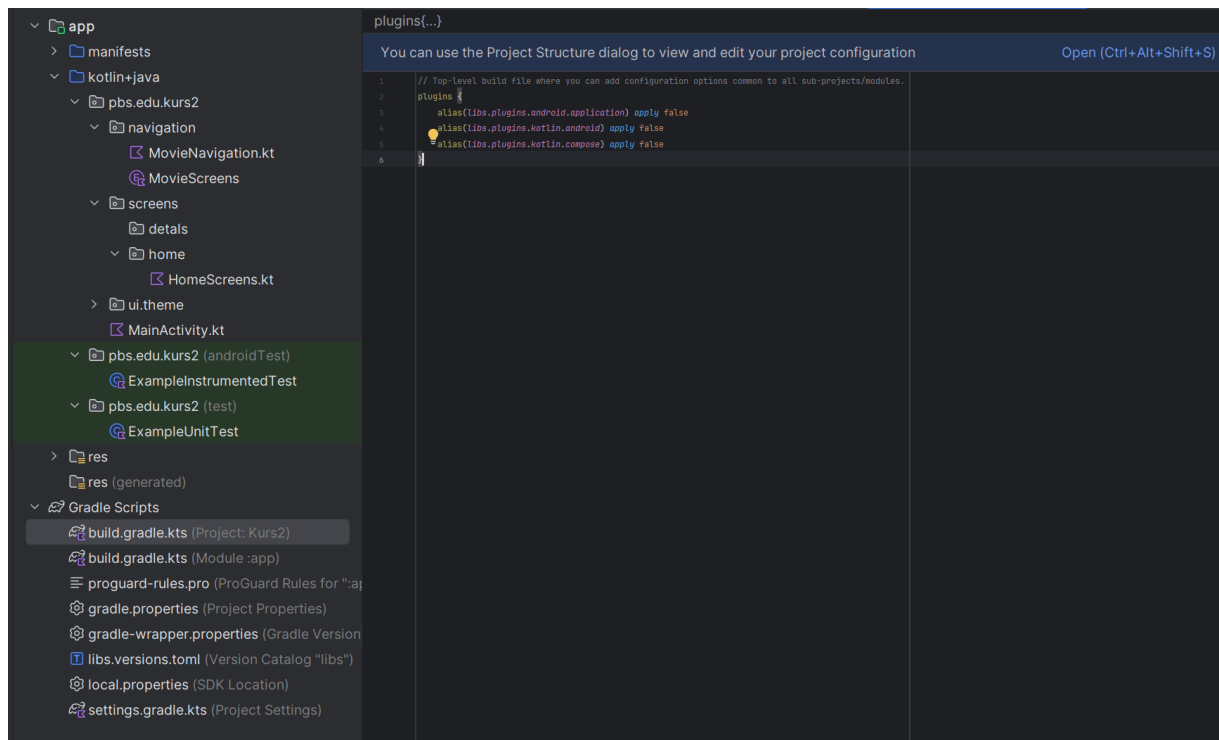
- Zawiera dwa ekrany:
 - HomeScreen – ekran główny z listą filmów.
 - DetailsScreen – ekran szczegółów wybranego filmu.
- Dzięki użyciu **enum**, nazwy ekranów są jednoznaczne i łatwe do odwołania w kodzie.
- ◊ **companion object i metoda fromRoute**
 - Funkcja fromRoute(route: String?) służy do **mapowania nazwy trasy (route)** na odpowiedni ekran.
 - Działa w następujący sposób:
 1. Pobiera część trasy **przed znakiem /** (substringBefore("/")).
 - To ważne, bo w trasie mogą być dodatkowe parametry, np. DetailsScreen/Avatar.
 2. Porównuje wynik z nazwami ekranów:
 - Jeśli to "HomeScreen" → zwraca MovieScreens.HomeScreen.
 - Jeśli to "DetailsScreen" → zwraca MovieScreens.DetailsScreen.
 - Jeśli route == null → domyślnie zwraca HomeScreen.
 - W innym przypadku rzuca wyjątek IllegalArgumentException.



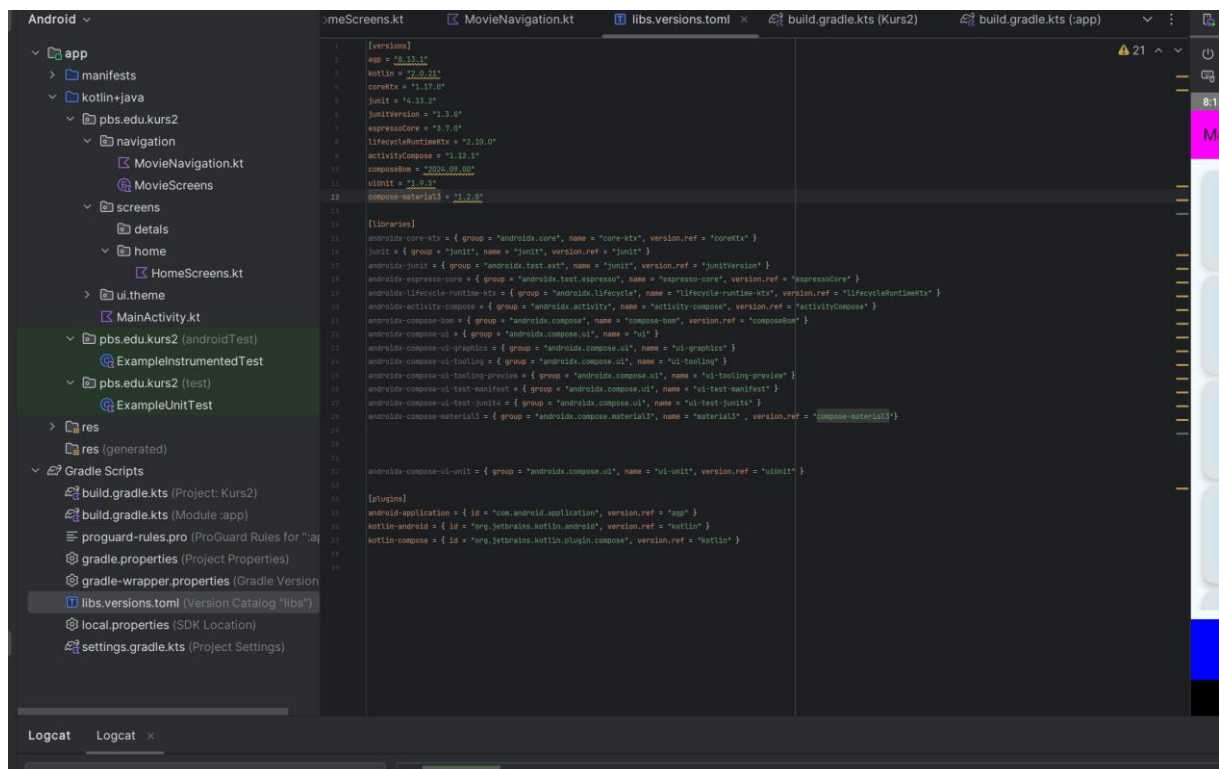
Build.gradle cz 1(Module)



Build.gradle cz 2(Module)



Build.gradle(projekt)



Plik libs.versions.toml

